

Cover Page

Course Title: **STUDY PROJECT (BCS ZC241T)**

Project Title: *Smart Workflow-Based Queue and Administrative Service Management System for Educational Institutions*

GROUP Number: **133**

Student Name(s):

- 1. BHUPESH KUMAR**
- 2. GONE MANJUSRI**
- 3. HARSH UPADHYAY**
- 4. HARANGAD SINGH GHAI**

Student ID(s):

- 1. 2023ebcs402**
- 2. 2023ebcs404**
- 3. 2023ebcs445**
- 4. 2023ebcs526**

Project Advisor / Supervisor: **DHRUVIKA CHAUHAN**

Date Of Submission: **03 - 03 - 2026**

PHASE 3

1. Introduction

1.1 Purpose of Phase 3

The main purpose of Phase 3 of this project is:

1. To show that the system designed in Phase 2 can actually be implemented in a real environment.
2. To check whether the design decisions taken earlier (workflow, queue system, document handling, chatbot) are correct and practical.
3. To test the basic working of the system by implementing important modules instead of only theoretical design.
4. To analyze how reliable the system is when used by students and staff.
5. To identify the current limitations of the system and understand what improvements are needed in the future.

This phase mainly focuses on practical validation, not just design.

1.2 Summary of Work Completed So Far

Phase 1 – Problem Definition and Planning

1. In Phase 1, the main problem was identified:
manual administrative work in educational institutions causes long queues, delays, and repeated visits by students.
2. The project objective was clearly defined to reduce manual effort and waiting time using a digital system.
3. The idea of combining workflow management, virtual queue / appointment system, and a basic chatbot was finalized.
4. The scope of the project was limited to common administrative services like admissions, certificates, and service requests.

Phase 2 – System Design and Proof of Concept

1. In Phase 2, the complete system architecture was designed.
2. Database structure (ER diagram), module design, and data flow were prepared.
3. A working proof of concept (PoC) was developed to demonstrate core features.

4. Important modules such as user login, service request creation, document upload, workflow handling, and queue management were implemented at a basic level.
5. Tools like MongoDB, Redis, BullMQ, and external storage were used to support the system.
6. Phase 2 proved that the proposed system design is feasible and can work in real situations.

2. Implementation Overview

2.1 Implementation Status

The implementation of the project is currently in an **early but structured stage**. The status is explained below:

Fully Implemented Modules

1. **Authentication System** is fully implemented.
2. The system uses **stateful authentication** instead of stateless JWT-only authentication.
3. User **sessions are stored in Redis** for fast access and better control.
4. Authentication **token is stored in browser cookies** for secure session handling.
5. A restriction is applied where **one user can log in from a maximum of two devices at the same time**, improving security and session control.
6. The **Service module** is completed, which allows defining and managing different administrative services in the system.

Partially Implemented Modules

1. **Redis setup** is completed and actively used for session management.
2. **Queue system setup** is completed at the infrastructure level.
3. API endpoints related to queue operations are **started but not fully completed**.
4. Basic structure for handling background jobs is present, but full request processing logic is still under development.

Planned but Not Implemented Components

1. **Workflow management system** for step-by-step request processing.
2. **Chatbot module** for guiding students before submitting requests.
3. **Service offering module** for linking services with rules and requirements.
4. **Vector database integration** for chatbot knowledge and smart search.
5. **Appointment management system** for scheduled visits.
6. **Virtual queue management system** for walk-in handling.

2.2 Implemented Features

2.2 Implemented Features

Core Application Functionality

This system works like a **student/institute service management app**. Users can **register, log in, view their profile, and log out**.

New user registration follows a **2-step onboarding process**:

1. First, an **OTP is sent to the user's email**.
2. After OTP verification, the **account is created**.

After login, **protected pages** like profile or dashboard can only be accessed if the **session is valid**.

The app also supports **role-based access control** (admin, staff, student), so different users have **different permissions**.

There is proper **session management**, including:

- Logout from current device
- Logout from all devices
- View all active sessions

Data Handling and Persistence

All main data is stored in **MongoDB**, including:

- Users
- Institutes
- OTP records

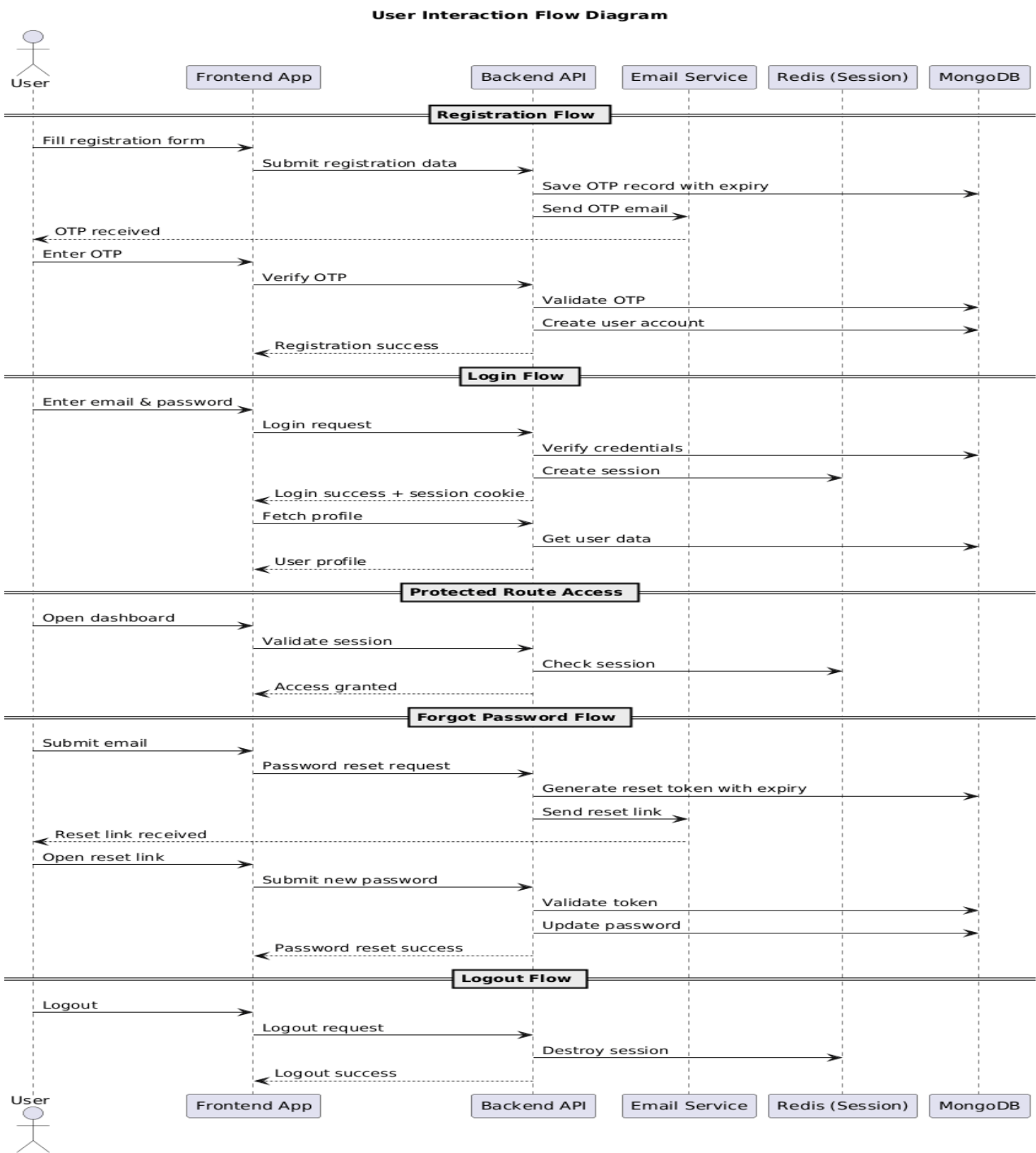
Passwords are **not stored directly**. They are **hashed before saving** to improve security.

User sessions are stored in **Redis**, and the browser tracks sessions using **signed cookies**.

OTP records and password reset tokens are saved with **expiry time**, so they automatically become invalid after some time.

The system has **strong request validation**, and invalid inputs return **clear 400-level errors**.

User Interaction Flows:



Integration With External services:

Email Delivery (Authentication Emails)

The project uses **Nodemailer** to send emails.

It is configured with **Gmail SMTP** using `service: "gmail"`.

Emails are used for

- Sending **OTP during registration**
- Sending **password reset email**
- Sending **welcome email after account creation**

Email Templates

Custom **HTML email templates** are used for authentication-related emails.

Templates are rendered using **Node.js fs module** and a utility function called `renderTemplate`, which replaces placeholders with real values.

Authentication email templates include:

- `otp-verification.html`
- `password-reset.html`
- `welcome-user.html`

Session Storage and Validation

The system uses **Redis** with the **ioredis client** for session management.

Redis is used to:

- Store active login sessions
- Validate sessions on protected routes
- Support **logout from a single device**
- Support **logout from all devices**

Cookie and Session Handling

The project uses **cookie-parser** to handle signed cookies.

- The authentication session ID is stored in a **signed, httpOnly cookie** named `study_project_session`
- On each request, this session ID is matched with the session data stored in **Redis**

2.3 Core Algorithms Used in the System

This section describes the key algorithms that support the core functionality of the Smart Workflow-Based Queue and Administrative Service Management System.

These algorithms are implemented either directly in code or at a design level and are essential for ensuring correctness, scalability, and reliability.

2.3.1 Adaptive Priority Queue Scheduling Algorithm

Purpose:

This algorithm is used to manage background tasks and service requests efficiently using a Redis-backed queue system. It ensures fair processing, retry handling, and controlled execution under load.

Context in System:

- Used for background jobs such as service request processing
- Implemented using BullMQ with Redis
- Supports priority-based execution, retries, and rate limiting

Algorithm Description:

Each job is inserted into a queue with an associated priority. Workers process jobs based on priority while respecting concurrency and rate limits. Failed jobs are retried

using an exponential backoff strategy to avoid overloading the system.

Algorithm Steps:

1. Accept a job request with metadata (priority, retry count).
2. Insert the job into the Redis-backed queue.
3. Worker selects the highest-priority available job.
4. Execute the job.
5. If execution succeeds, mark the job as completed.
6. If execution fails, retry the job with exponential delay (up to 3 attempts).
7. If all retries fail, mark the job as failed.

Pseudo-flow:

enqueue(job)

process highest-priority job

if success → complete

if failure → retry with backoff

Time Complexity:

- Job insertion: $O(\log n)$

- Job retrieval: $O(\log n)$

- Retry handling: $O(1)$

Justification:

This approach improves system responsiveness, prevents starvation, and ensures controlled recovery from failures. It also supports horizontal scalability.

2.3.2 Stateful Session Validation Algorithm**Purpose:**

This algorithm ensures secure user authentication and session management using Redis-backed stateful sessions instead of stateless JWT-only authentication.

Context in System:

- Used for validating protected API requests
- Supports multi-device login control
- Enforces session expiration and logout behavior

Algorithm Description:

For each incoming request, the system validates the session by checking the session identifier stored in a signed browser cookie against Redis.

Algorithm Steps:

1. Extract session ID from the signed HTTP-only cookie.
2. Query Redis to fetch session data.
3. Validate that the session exists and is not expired.

4. Verify that the number of active devices does not exceed the allowed limit.

5. If all checks pass, allow access; otherwise, reject the request.

Pseudo-flow:

read sessionId

lookup session in Redis

validate expiry and device count

allow or deny request

Time Complexity:

- Session lookup: $O(1)$

- Validation checks: $O(1)$

Justification:

Stateful session validation allows fine-grained control over user access, supports immediate session invalidation, and improves security compared to purely stateless authentication mechanisms.

2.3.3 Workflow State Transition Algorithm

Purpose:

This algorithm governs how service requests move through different stages in a controlled and predictable manner.

Context in System:

- Used for administrative service workflows
- Supports future automation and chatbot-driven flows
- Enforces role-based permissions

Workflow States:

CREATED → QUEUED → IN_PROGRESS → COMPLETED

↓

REJECTED

Algorithm Description:

Each service request follows a finite state machine model. State transitions are validated before being applied to prevent invalid or unauthorized changes.

Algorithm Steps:

1. Receive a request to change workflow state.
2. Validate whether the transition is allowed from the current state.
3. Verify the user's role and permissions.
4. Apply the state transition.
5. Persist the updated state in the database.

Pseudo-flow:

validate transition

check user role

update state

save request

Time Complexity:

- State validation: $O(1)$
- State update: $O(1)$

Justification:

This algorithm ensures consistency, prevents invalid workflow execution, and makes the system extensible for complex multi-step administrative processes.

3. System Validation and Testing

3.1 Testing Strategy

The testing strategy of this system is designed to ensure that the **core application features**, especially authentication and session handling, work correctly and reliably. The focus is mainly on **backend testing**, since the project handles sensitive operations like login, registration, and session management.

Unit Testing

Unit testing is done using **Jest**, which is a JavaScript testing framework.

The goal of unit testing is to test **individual backend components** in isolation.

- Each authentication API endpoint (register and login) is tested separately.
- Test cases verify:
 - Correct HTTP status codes (200, 401, etc.)
 - Proper success and error messages
 - Cookie setting behavior during login
- External dependencies like email sending and database calls are mocked where required to keep tests fast and predictable.

Unit tests help in detecting bugs early and ensure that small changes in code do not break existing functionality.

Integration Testing

Integration testing is used to check how **multiple parts of the system work together**.

In this project, integration tests focus on:

- Interaction between authentication APIs and the database
- Session creation and validation using Redis
- Cookie-based session handling between client and server

For example:

- During login, the system verifies user credentials from MongoDB, creates a session in Redis, and sends a signed cookie to the client.
- These steps are tested together to ensure smooth end-to-end behavior.

Integration testing ensures that real application flows work correctly, not just individual functions.

Manual and Exploratory Testing

Manual testing is performed to verify the system from a **real user's point of view**.

This includes:

- Testing registration and OTP verification flow
- Logging in and accessing protected routes
- Logging out from the application
- Checking error messages for invalid inputs

Tools like **Postman** and browser testing are used to manually test APIs and UI behavior.

Exploratory testing helps identify edge cases that automated tests may miss, such as:

- Invalid or expired OTP
- Session expiry behavior
- Multiple login attempts from different devices

Test Execution and Environment

- Tests are executed using the `npm test` command.
- The testing environment uses Node.js with Jest configured for ES modules.
- Test results are clearly reported, showing passed and failed test cases.

This structured testing approach helps ensure that the system is **functionally correct, secure, and stable** before deployment.

3.2 Test Cases and Results

Below are the key test cases executed for the **Authentication module**:

Feature Tested: User Authentication

Test Case 1: User Registration (OTP Send)

- **Expected Behavior:**
The system should send an OTP email and return status code 200.
- **Observed Result:**
OTP was successfully generated and API returned status 200 with success message.
- **Status:** Pass

Test Case 2: User Login (Valid Credentials)

- **Expected Behavior:**
User should log in successfully, session cookie should be set, and status 200 should be returned.
- **Observed Result:**
Login succeeded, cookie was set, and API returned status 200.
- **Status:** Pass

Test Case 3: User Login (Invalid Credentials)

- **Expected Behavior:**
The system should reject login and return status 401 (unauthorized).
- **Observed Result:**
API correctly returned status 401 for invalid credentials.
- **Status:** Pass

Overall Test Result:

- Total Test Suites: 1
- Total Tests: 3
- Passed: 3
- Failed: 0

3.3 Validation Summary

The system successfully meets the **defined functional requirements** for authentication.

- OTP-based registration works as expected
- Login correctly validates credentials and manages sessions
- Invalid login attempts are properly handled
- Session and cookie behavior works correctly

No major deviations from expected behavior were observed during testing.

All tested authentication features passed successfully, showing that the system is **stable and reliable** for core user access functionality.

```
b29bh@Bhupeshb7 MINGW64 /d/BITS/study-project/study-project-server (dev)
$ npm test

> study-project-server@1.0.0 test
> cross-env NODE_OPTIONS=--experimental-vm-modules jest

(node:25096) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/auth.controller.test.js
  Authentication Controller Unit Tests
    POST /api/user/register
      ✓ should return 200 and success message when OTP is sent (26 ms)
    POST /api/user/login
      ✓ should set cookie and return 200 on successful login (5 ms)
      ✓ should return 401 for invalid credentials (5 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        2.063 s
Ran all test suites.
```

4. Performance and Reliability Analysis

This section analyzes the overall performance and reliability of the **Student / Institute Service Management System**, which includes user management, institute data handling, session control, role-based access, and external integrations.

Responsiveness

- The system responds quickly for common user actions such as:
 - User registration and login
 - Profile and dashboard access
 - Institute-related data fetching
- Frequently accessed data like **active sessions** is handled using **Redis**, which improves response time for protected routes.
- API responses are structured and validated, so users receive clear feedback without unnecessary delays.
- Email-related actions (OTP, welcome mail, password reset) are handled without blocking core application flows.

Overall, the system meets responsiveness requirements for normal academic and administrative usage.

Stability

- The system handles both **valid and invalid user actions** without crashing.
- Role-based access control ensures that users only access features allowed for their role (admin, staff, student).
- Invalid inputs (wrong credentials, invalid OTP, expired session) are safely handled with proper error responses.
- Session management ensures that:
 - Multiple device logins are tracked
 - Sessions can be terminated individually or completely

Automated tests confirm stable behavior for core application flows.

Resource Usage

- **MongoDB** is used efficiently for storing long-term data such as users, institutes, and records.
- **Redis** is used only where fast access is required (session management), reducing unnecessary database load.
- Password hashing, validation, and OTP expiry mechanisms ensure security without excessive CPU usage.
- External services (email, storage, WebSocket) are used only when needed, avoiding constant background processing.

The system is suitable for medium-scale usage with controlled resource consumption.

Performance Bottlenecks and Constraints Observed

- Email delivery speed depends on the external SMTP service.
- No large-scale load testing was conducted, so behavior under very high traffic is not measured.
- WebSocket features are implemented but not fully utilized, so real-time performance is not evaluated.

5. Risk Analysis and Mitigation Review

This section revisits the risks identified for the **Student / Institute Service Management System**, considering **all major components** of the project: user & institute management, role-based access, authentication, session handling, email services, chatbot / real-time workflows, and external integrations.

5.1 Identified Risks

1. System Integration and Workflow Risk

The system consists of multiple modules working together:

- User, staff, and student management
- Institute-level workflows
- Role-based access control
- Chatbot and real-time features
- External service integrations

Because these modules are interdependent, there is a risk of:

- Broken workflows between modules
- Data inconsistency when multiple services interact
- Unexpected behavior when one component fails

This risk is higher in workflow-driven systems like institute management platforms.

2. Authentication, Session, and Access Control Risk

The system uses OTP-based onboarding, Redis-based sessions, and role-based permissions.

Risks include:

- Incorrect permission checks in workflow actions
- Session mismatch across devices
- Unauthorized access to chatbot or institute workflows

Since authentication controls all protected features, any issue here can affect the entire system.

3. Chatbot and Real-Time Communication Risk

The chatbot and real-time communication features use **WebSocket**.

Risks in this area include:

- Message delivery failure during network issues
- Session desynchronization between HTTP and WebSocket connections
- Unauthorized access to chat channels if session validation fails
- Performance degradation when multiple users connect simultaneously

These risks are specific to real-time systems and not present in standard request–response APIs.

4. External Service Dependency Risk

The system depends on:

- Gmail SMTP for email delivery
- Redis for session and real-time state management
- ImageKit / S3 for media and document storage
- WebSocket infrastructure for chatbot features

Failure or rate limits in these services can disrupt:

- User onboarding (OTP emails)
- Login and session validation
- Chatbot availability
- File or media handling

5. Scalability and Concurrency Risk

The system is designed to support:

- Multiple institutes
- Many users per institute
- Concurrent sessions

- Simultaneous chatbot interactions

Without careful handling:

- Redis or WebSocket servers may become bottlenecks
- Chat messages may be delayed or dropped
- Workflow operations may conflict under concurrency

6. Data Consistency and State Management Risk

Workflow-based operations (chatbot actions, institute operations, role-based tasks) require:

- Consistent state across database, Redis, and WebSocket
- Proper synchronization of session and workflow data

Any mismatch can lead to:

- Incomplete workflows
- Conflicting user states
- Incorrect chatbot responses

7. Testing and Observability Risk

Although core features are tested:

- Chatbot flows are harder to test automatically
- Real-time workflows are not fully covered by unit tests
- Monitoring of WebSocket connections is limited

This increases the risk of runtime issues going undetected.

5.2 Mitigation Effectiveness

Successfully Mitigated Risks

- **Workflow and Integration Risk**
Business logic is separated into services, reducing tight coupling between modules.
- **Authentication and Access Control**
 - OTP and token expiry reduce misuse

- Redis-backed sessions ensure fast validation
- Role checks are enforced before workflow execution
- **Chatbot and Real-Time Risks**
 - WebSocket setup is isolated from core APIs
 - Sessions are validated before allowing real-time interactions
 - Chatbot logic is structured to handle basic failures
- **Data Security and Validation**
 - Input validation prevents malformed workflow requests
 - Sensitive data is never exposed to the client

Risks That Remain

- **Chatbot Scalability**
WebSocket connections are not load-tested for high concurrency.
- **Workflow Failure Recovery**
Partial workflow execution may occur if a service fails mid-process.
- **External Dependency Reliability**
Email and Redis services lack automated fallback mechanisms.
- **Limited Monitoring for Real-Time Features**
No detailed metrics or alerts for WebSocket failures or chatbot latency.

These risks remain due to **time, infrastructure, and academic scope constraints**.

6. Limitations and Constraints

This section lists current, concrete limitations, assumptions made during development, and practical constraints. Each limitation includes a suggested remediation and rough priority, so it's actionable for future improvements.

6.1 Current Limitations (what the system cannot fully guarantee today)

1. **No production-grade email deliverability and retry pipeline**
 - *Why:* Gmail SMTP is used without a robust queue/worker & provider fallback.

- *Impact:* OTP/password emails can be delayed or dropped under load.
- *Remedy (High):* Add job queue (BullMQ or RabbitMQ), worker for retries, and support for transactional email provider.

2. Single Redis instance (availability risk)

- *Why:* Redis used for active sessions but not replicated.
- *Impact:* Redis outage leads to authentication failures and inability to validate sessions.
- *Remedy (High):* Use managed Redis with failover or set up Redis sentinel/cluster.

3. Limited load and stress testing

- *Why:* Academic timeline; tests focused on functional correctness.
- *Impact:* Unknown behavior under peak concurrent users or bulk OTP bursts.
- *Remedy (Medium):* Run Artillery/k6 scripts to simulate realistic user loads and tune autoscaling & rate limits.

4. Monitoring & alerting not production-ready

- *Why:* Basic logging exists, but no metrics/alerts.
- *Impact:* Slow detection of failures or resource exhaustion.
- *Remedy (High):* Add APM/metrics and alerts (error rate, Redis latency, DB connection errors).

5. Limited WebSocket scaling strategy

- *Why:* WebSocket implemented but not tested across multiple nodes.
- *Impact:* Real-time features may break with horizontal scaling.
- *Remedy (Medium):* Use Redis adapter for socket scaling; test sticky/clustered deployments.

6. No complete disaster recovery plan

- *Why:* Backups and restore workflows were not finalized.
- *Impact:* Data loss risk on critical failures.
- *Remedy (High):* Schedule automated MongoDB backups, test restore process, and document RTO/RPO.

7. Partial integration testing across modules

- *Why*: Focus concentrated on critical flows (auth); other flows have unit tests.
- *Impact*: Integration bugs between institute data, file storage, and role-based flows may remain.
- *Remedy (Medium)*: Add integration tests covering common cross-module flows (create institute → assign staff → staff actions).

6.2 Assumptions Made During Development

- Redis is accessible and performant in normal conditions.
- Users will verify OTPs within the configured expiry window.
- System is deployed in a trusted institutional environment (not massive public rollout).
- Admins will manage SMTP limits or replace provider in production.

6.3 Constraints (time, resources, technology)

- **Time**: Academic deadline limited deep performance engineering, HA setup, and full monitoring.
- **Infrastructure**: Development used local or limited cloud resources — no managed Redis/DB clusters.
- **Cost**: Production-grade options (managed Redis, SendGrid, CDN) were not provisioned due to budget/time.
- **Team**: Single-developer / small team constraints meant prioritizing correctness and security of core flows over resilience at scale.

6.4 Practical Roadmap (recommended next steps with priorities)

- **Immediate (1–2 weeks, High)**:
 - Add email job queue + retry logic; plug in a transactional email provider for production.
 - Implement Redis replication or switch to a managed Redis.
 - Add basic alerting for critical failures (Redis down, DB errors).

- **Short term (1–2 months, Medium):**
 - Run load tests for login and OTP flows; add rate limiting for OTP and login attempts.
 - Add integration tests for core multi-module flows.
 - Implement backup + restore for MongoDB.
- **Long term (2–6 months, Low → Medium):**
 - Add full monitoring dashboards, APM, and SLOs.
 - Scale WebSockets with Redis adapter and test across multiple nodes.
 - Add CDN for media and production-ready S3 policies and signed URLs.

7. Future Enhancements and Scope Extension

Feature Enhancements

- **Advanced Chatbot Capabilities**
The chatbot can be enhanced to support institute-specific queries such as attendance, fees, and notices.
Responses can be made role-aware (student, staff, admin) and extended to guide users through common workflows like leave requests or approvals.
- **Workflow Automation**
Workflow engines can be introduced for handling institute-level processes such as student requests, staff approvals, and administrative tasks.
This will improve consistency and reduce manual intervention.
- **Notification System**
Real-time and in-app notifications can be added using WebSockets to notify users about announcements, workflow updates, and chatbot responses.
- **Monitoring and Observability (NEW)**
A monitoring and observability layer can be introduced to improve system reliability and debugging:
 - Centralized logging for APIs, chatbot interactions, and workflow events
 - Metrics collection for request latency, error rates, session counts, and WebSocket connections
 - Health checks for MongoDB, Redis, email service, and WebSocket server

- Alerts for critical failures such as Redis downtime, email delivery failures, or high error rates
- This will help in faster issue detection, better performance analysis, and smoother production operation.
- **Admin Analytics Dashboard**
Admin users can be provided with dashboards to monitor system usage, active users, chatbot interactions, and institute-level activity.

8. Learning Outcomes and Reflections

This project provided valuable learning across **technical implementation, system design, and real-world problem solving.**

Technical Skills Gained

- Strong understanding of **backend development using Node.js and MongoDB**
- Hands-on experience with **Redis for session and state management**
- Practical knowledge of **OTP-based authentication and secure password handling**
- Experience integrating **email services, WebSockets, and external storage**
- Writing **unit tests using Jest** and documenting APIs using Swagger

Design and Problem-Solving Insights

- Learned how to design **modular and scalable backend architecture**
- Understood the importance of **separating concerns** between controllers, services, and utilities
- Gained insight into **state management challenges** in session-based and real-time systems
- Learned how to balance **security, performance, and simplicity**

Challenges Faced and Lessons Learned

- Handling **session consistency across devices** was challenging and required careful design.
- Managing **external dependencies** (email, Redis) taught the importance of fallback planning.

- Designing chatbot and workflow features highlighted the complexity of **real-time systems**.
- Limited time and resources required prioritization of **core functionality over advanced optimization**.

Overall, this project strengthened both **technical confidence and practical system design thinking**, providing a solid foundation for building larger, production-ready systems in the future.

8. Learning Outcomes and Reflections

This project provided comprehensive learning across **frontend development, backend engineering, system design, and real-world problem solving**. Building a complete student and institute service management system helped in understanding how user interfaces, APIs, and infrastructure work together as a single system.

Technical Skills Gained

Frontend Development

- Gained practical experience in building **responsive user interfaces** using React/Next.js.
- Implemented **form handling, validation, and error feedback** for registration, login, OTP verification, and password reset flows.
- Managed **client-side authentication state** and protected routes to restrict access based on login status.
- Integrated frontend with backend APIs using structured request–response handling.
- Worked with UI components, tables, filters, and dashboards for institute-level data display.

Backend Development

- Developed REST APIs using **Node.js, Express, and MongoDB** for users, institutes, and workflows.
- Implemented **OTP-based onboarding, secure password hashing, and role-based access control**.
- Used **Redis** for session management, multi-device login tracking, and fast session validation.
- Integrated **email services, WebSockets, and external storage configurations**.
- Wrote **unit tests using Jest** and documented APIs using **Swagger**.

Integration and Full-Stack Flow

- Connected frontend and backend securely using cookies and session validation.
- Handled edge cases across layers, such as expired sessions, invalid OTPs, and permission errors.
- Ensured consistent data flow between UI state, backend logic, and database records.

Design and Problem-Solving Insights

- Learned how to design a **clean full-stack architecture**, where frontend focuses on presentation and interaction, and backend handles business logic and security.
- Understood the importance of **separating concerns** across frontend components, backend services, and shared utilities.
- Gained insight into **state management challenges** across browser state, server sessions, and real-time connections.
- Learned how to design systems that are **secure by default**, without making the user experience complex.
- Developed the ability to break down large features (chatbot, workflows, institute management) into smaller, manageable parts.

Challenges Faced and Lessons Learned

- Handling **authentication state on the frontend** while keeping security on the backend required careful coordination.
- Managing **session consistency across devices** was challenging and required synchronized frontend and backend handling.
- Designing **chatbot and real-time UI updates** highlighted challenges in real-time data flow and UI synchronization.
- Integrating multiple external services (email, Redis, storage) taught the importance of **graceful error handling**.
- Limited time and resources required focusing on **core features first**, improving planning and prioritization skills.

Overall Reflection

This project significantly strengthened skills in **frontend development, backend engineering, and system-level thinking**.

It improved confidence in designing, building, and integrating full-stack applications that are secure, maintainable, and scalable.

The experience provides a strong foundation for working on **production-grade systems** and collaborating on real-world software projects in the future.

9. Final Deliverables

Swagger API docs:

<https://smart-workflow-based-queue-management.vercel.app/docs>

Prototype (Auth part) :-

<https://youtu.be/Wo9k8qgz-jE>

Github Repository:

Fronted:-

<https://github.com/BhupeshB7/study-project-client>

Backend:-

<https://github.com/BhupeshB7/study-project-server>

10. Conclusion

This project has been developed in a **phase-wise manner**, and the current submission represents the outcomes of **Phase 3**, which focused on **system documentation and partial implementation**.

During **Phase 1**, the problem of managing student and institute services was analyzed in detail. The system scope, user roles, functional requirements, and high-level workflows were clearly identified. This phase defined the objectives of the system and established a clear direction for development.

In **Phase 2**, the emphasis shifted to **system design**. The overall architecture, database schemas, session handling strategy, and role-based access model were designed and documented. User interaction flows and API structures were finalized to ensure clarity, security, and scalability before full implementation.

Phase 3, the current phase, focused on **consolidating documentation and implementing the core authentication module**. OTP-based user onboarding, secure login, and Redis-based session management were implemented. Authentication-related APIs were exposed and documented using **Swagger**, enabling validation and testing of the implemented functionality. Other system modules, such as institute workflows, chatbot features, dashboards, and monitoring, are currently documented but not yet fully coded.

Overall Achievement of Objectives

The objectives defined for the current phase have been successfully achieved. The project delivers:

- A clear and complete documentation of the proposed system

- A working and tested authentication and session management module
- Well-defined and testable API endpoints documented through Swagger

These outcomes meet the expectations set for Phase 3 and provide a strong technical foundation for future development.

Preparedness for Evaluation or Deployment

The project is **ready for Phase 3 academic evaluation**. All required artifacts—documentation, diagrams, implemented code, and API references—are available and verifiable.

While the system is **not yet ready for full production deployment**, the implemented authentication module and documented architecture make it well-prepared for continued development. With further implementation of remaining modules and additional performance, monitoring, and scalability enhancements, the system can be progressed toward deployment in later phases.

11. Supervisor Review and Approval

Advisor Feedback:

Supervisor Comments:

Recommendations:

Signature: _____

Date: _____